arquitecturas web

bloque 2.2: servidores web, de aplicaciones y despliegue

1. Contenido estático vs. dinámico

De las arquitecturas al "dónde vive" todo esto

- En el bloque 2.1 vimos **cómo se organiza** una aplicación web (capas, microservicios, MVC, Hexagonal...).
- Hoy vemos **dónde vive físicamente** ese código y **quién lo sirve** cuando llega una petición HTTP.

Contenido estático

- Ficheros que el servidor envía **tal cual están guardados en disco**, sin ejecutar ningún código por petición.
- Ejemplos: HTML fijo, CSS, JavaScript del cliente, imágenes, PDFs, fuentes.
- Muy rápido de servir: el servidor solo lee el fichero y lo envía.

Contenido dinámico

- El servidor **ejecuta código** en cada petición para generar la respuesta (p. ej. consultar una base de datos y montar el HTML al vuelo).
- Ejemplos: el resultado de una búsqueda, el precio actualizado de un producto, el feed personalizado de un usuario.
- Más lento que el estático (hay que ejecutar lógica), pero imprescindible para cualquier aplicación con datos que cambian.

Una misma página puede mezclar ambos

```
GET /tienda  →  HTML generado dinámicamente (precios, stock)
                que a su vez referencia:
                /css/estilos.css   → estático
                /img/logo.png      → estático
                /js/analytics.js   → estático
```

Lo habitual es servir lo estático de la forma más directa posible y reservar el coste de "ejecutar código" solo para lo que de verdad cambia.

Repaso — Sección 1, Pregunta 1

¿Qué caracteriza al contenido estático?

A) Se genera ejecutando una consulta a base de datos en cada petición
B) Es siempre más lento de servir que el contenido dinámico
C) Se envía tal cual está guardado en disco, sin ejecutar código por petición
D) Solo puede ser HTML, nunca imágenes ni CSS

Repaso — Sección 1, Pregunta 1 (solución)

✅ **C)** Se envía tal cual está guardado en disco, sin ejecutar código por petición

Repaso — Sección 1, Pregunta 2

¿Cuál de estos recursos es un ejemplo típico de contenido dinámico?

A) Un fichero `logo.png`
B) El resultado personalizado de una búsqueda en una tienda online
C) Una hoja de estilos `.css`
D) Una fuente tipográfica `.woff`

Repaso — Sección 1, Pregunta 2 (solución)

✅ **B)** El resultado personalizado de una búsqueda en una tienda online

Repaso — Sección 1, Pregunta 3

¿Por qué una misma página web puede combinar contenido estático y dinámico a la vez?

A) No es posible, una página es siempre 100% estática o 100% dinámica

B) Porque el HTML principal puede generarse dinámicamente mientras referencia recursos estáticos como CSS, JS o imágenes

C) Porque el navegador decide aleatoriamente qué partes son estáticas

D) Porque HTTP lo prohíbe expresamente

Repaso — Sección 1, Pregunta 3 (solución)

✅ **B)** Porque el HTML principal puede generarse dinámicamente mientras referencia recursos estáticos como CSS, JS o imágenes

2. Renderizado, CDN y SEO

¿Dónde se "monta" el HTML que ve el usuario?

Hasta ahora hemos hablado de "generar HTML dinámico" como si fuera una sola cosa. En realidad, importa mucho **dónde** se monta ese HTML: en el servidor o en el navegador del usuario.

CSR: Client-Side Rendering

- El servidor envía un HTML casi vacío (un `<div id="app"></div>`) junto con un paquete de JavaScript grande.
- El **navegador** ejecuta ese JavaScript, que construye el HTML real y lo pinta en pantalla.
- Típico de SPA (Single Page Applications) hechas con React, Vue o Angular sin configuración adicional.
- Ventaja: navegación muy fluida entre páginas una vez cargado. Inconveniente: la primera carga tarda más, y la página está "en blanco" hasta que el JS termina de ejecutarse.

SSR: Server-Side Rendering

- El **servidor** ejecuta la lógica de la aplicación y genera el HTML **ya completo** en cada petición, igual que hacían los servidores de aplicaciones "clásicos" (PHP, JSP...).
- El navegador recibe una página ya pintada, visible de inmediato, aunque el JavaScript todavía no se haya cargado del todo.
- Frameworks modernos como **Next.js** (React) o **Nuxt** (Vue) permiten hacer SSR combinando lo mejor de ambos mundos.

Hidratación (hydration)

- Técnica habitual en SSR moderno: el servidor manda el HTML ya renderizado, y **después** el navegador "engancha" el JavaScript a ese HTML para que la página vuelva a ser interactiva (clics, formularios...), sin tener que redibujarla desde cero.
- Es el punto intermedio entre SSR puro y CSR puro: carga inicial rápida (como SSR) + interactividad rica después (como CSR).

SSG: Static Site Generation (de paso)

- Variante de SSR en la que el HTML **no se genera en cada petición**, sino una sola vez, en tiempo de **build** (antes de desplegar).
- El resultado es, en la práctica, contenido **estático**: ideal para contenido que no cambia por usuario (un blog, una documentación).
- Si el contenido cambia (precios, sesión de usuario…), SSG no sirve por sí solo: hace falta SSR o contenido dinámico real.

¿Por qué importa esto para el SEO?

- Los buscadores (Google y otros) usan "crawlers" (rastreadores) que visitan tus páginas para indexarlas.
- Un crawler que recibe un CSR puro ve, al principio, un HTML casi vacío: tiene que **ejecutar el JavaScript** para ver el contenido real, lo cual no todos los rastreadores hacen igual de bien ni con la misma rapidez.
- SSR (o SSG) entrega el contenido **ya visible en el HTML inicial**, lo que facilita muchísimo el indexado correcto y rápido.

SEO: no es solo el crawler

Además del renderizado, el SEO depende de cosas que sí dependen de ti:

- Etiquetas `<title>` y `<meta description>` bien escritas por página.
- URLs legibles y estables (evitar que cambien sin redirección `301`).
- Tiempos de carga bajos: Google penaliza páginas lentas en el ranking.
- Contenido accesible sin JavaScript imprescindible para lo esencial (texto, enlaces).

CDN: Content Delivery Network

- Una **CDN** es una red de servidores distribuidos geográficamente que guardan copias cacheadas de tu contenido, cerca de cada usuario.
- Un usuario en Tokio no tiene por qué ir hasta tu servidor en Madrid: la CDN le sirve una copia cacheada desde un nodo cercano.
- Ejemplos conocidos: **Cloudflare**, **Akamai**, **Fastly**, **Amazon CloudFront**.

Qué beneficia más a una CDN

- El contenido **estático** (CSS, JS, imágenes, incluso HTML generado por SSG) es el más fácil de cachear: no cambia por usuario.
- El contenido dinámico y personalizado (tu carrito, tu sesión) normalmente **no** se cachea en la CDN, salvo técnicas más avanzadas que no veremos aquí.
- Por eso separar bien estático de dinámico (sección 1) también ayuda a sacarle partido a una CDN.

El problema de invalidar la caché

- Si despliegas una nueva versión de `estilos.css` pero la CDN (o el propio navegador) sigue sirviendo la copia antigua cacheada, los usuarios no ven el cambio.
- Solución habitual: **cache busting**, incluir un hash o versión en el nombre del fichero (`estilos.a3f9c1.css`): al cambiar el contenido, cambia el nombre, y la caché antigua deja de usarse automáticamente porque la URL es distinta.

Repaso — Sección 2, Pregunta 1

¿Qué diferencia principal hay entre CSR y SSR?

A) No hay ninguna diferencia, son sinónimos
B) En CSR el navegador construye el HTML ejecutando JavaScript; en SSR el servidor genera el HTML ya completo antes de enviarlo
C) CSR solo puede usarse con PHP y SSR solo con React
D) SSR no permite usar JavaScript en ningún momento

Repaso — Sección 2, Pregunta 1 (solución)

✅ **B)** En CSR el navegador construye el HTML ejecutando JavaScript; en SSR el servidor genera el HTML ya completo antes de enviarlo

Repaso — Sección 2, Pregunta 2

¿Por qué una aplicación CSR pura puede tener problemas de SEO?

A) Porque CSR siempre está prohibido por los buscadores

B) Porque el HTML inicial que recibe el crawler puede estar casi vacío hasta que se ejecuta el JavaScript

C) Porque CSR no puede usar HTTPS

D) Porque CSR solo funciona en dispositivos móviles

Repaso — Sección 2, Pregunta 2 (solución)

✅ **B)** Porque el HTML inicial que recibe el crawler puede estar casi vacío hasta que se ejecuta el JavaScript

Repaso — Sección 2, Pregunta 3

¿Qué tipo de contenido aprovecha mejor una CDN?

A) El contenido dinámico y personalizado de cada usuario, como su carrito

B) El contenido estático (CSS, JS, imágenes) o generado una sola vez (SSG), al no cambiar por usuario

C) Ninguno, las CDN no cachean nada

D) Solo los vídeos, nunca el HTML ni el CSS

Repaso — Sección 2, Pregunta 3 (solución)

✅ **B)** El contenido estático (CSS, JS, imágenes) o generado una sola vez (SSG), al no cambiar por usuario

3. Servidor web vs. servidor de aplicaciones

Servidor web

- Programa cuya función principal es **recibir peticiones HTTP y responder**, típicamente sirviendo contenido estático con muchísima eficiencia.
- Ejemplos: **Apache HTTP Server**, **Nginx**.
- También puede actuar de "puerta de entrada" (proxy inverso) hacia otros servicios, aunque eso lo veremos a fondo en la UD3.

Otra optimización típica del servidor web: compresión

- Antes de enviar contenido (sobre todo texto: HTML, CSS, JS, JSON), el servidor web puede comprimirlo con **gzip** o, más modernamente, **brotli**.
- El navegador indica qué compresiones acepta con la cabecera `Accept-Encoding` (vista en el bloque 2.1); el servidor responde con `Content-Encoding: gzip` (o `br`) y el cuerpo comprimido.
- Reduce mucho el tamaño transferido sin tocar ni una línea de la aplicación: es una configuración del servidor web, no del código.

Servidor de aplicaciones

- Programa diseñado para **ejecutar la lógica de la aplicación** (el código de negocio) y generar contenido dinámico.
- Ejemplos: **Tomcat** (Java), **Node.js** ejecutando una app Express, **Gunicorn/uWSGI** (Python), **PHP-FPM**.
- Entiende el lenguaje/framework de la aplicación; un servidor web puro no sabe "ejecutar" ese código por sí mismo.

Frameworks de backend populares, por lenguaje

Cada framework de backend necesita, por detrás, un servidor de aplicaciones que lo ejecute:

Lenguaje	Framework	Servidor de aplicaciones típico
Java	**Spring** (Spring Boot)	Tomcat (a menudo embebido dentro del propio `.jar`)
Python	**Django**, Flask	Gunicorn o uWSGI
PHP	**Laravel**, **Symfony**	PHP-FPM
Node.js	**Express**, NestJS	El propio proceso Node.js
Ruby	Ruby on Rails	Puma o Unicorn

¿Y Vue, Angular o React? Esos son frontend

- **Vue**, **Angular** y **React** son frameworks de **frontend**: su código se ejecuta en el **navegador** (CSR, visto en la sección 2), no en un servidor de aplicaciones.
- No "sustituyen" a Spring/Django/Laravel: conviven con ellos. Lo habitual es un backend (p. ej. Laravel) exponiendo una API, y un frontend (p. ej. Vue) consumiéndola desde el navegador.

Entonces, ¿quién sirve ese frontend? Un servidor web

- El comando de build (`npm run build`) de Vue/Angular/React **no genera un servidor**: genera una carpeta con ficheros `index.html` , `.js` y `.css` ya compilados — es decir, **contenido estático** (sección 1).
- Ese resultado se sirve exactamente igual que cualquier otro estático: con **Nginx o Apache**, copiando esa carpeta a la raíz que el servidor web expone.
- Por eso un SPA en CSR no necesita Node.js en producción para "ejecutarse": Node.js solo hizo falta en tiempo de **build**, en tu máquina (o en el pipeline de CI/CD, UD5).
- Caso distinto: si ese frontend usa **SSR** (Next.js para React, Nuxt para Vue), entonces sí hace falta un servidor de aplicaciones de verdad ejecutándose en producción: un proceso **Node.js**, porque el HTML se genera en cada petición, no de una vez en el build.

CORS: qué pasa si frontend y backend viven en orígenes distintos

- Si tu frontend estático se sirve desde `https://miapp.com` y tu API vive en `https://api.miapp.com` (u otro puerto), el navegador los trata como **orígenes distintos**.
- Por seguridad, el navegador bloquea por defecto que el JavaScript de un origen llame a un origen distinto, salvo que el servidor lo permita explícitamente: es la política del **mismo origen** (*same-origin policy*).
- **CORS** (Cross-Origin Resource Sharing) es el mecanismo con el que el backend indica, mediante cabeceras HTTP, qué orígenes externos tiene permitido llamarle.

CORS en la práctica

- El servidor de aplicaciones añade cabeceras como `Access-Control-Allow-Origin: https://miapp.com` a sus respuestas.
- Sin esa cabecera (o con un valor que no coincide), el navegador **descarta la respuesta** antes de entregársela al código JavaScript, aunque la petición haya llegado bien al servidor.
- Es uno de los errores más comunes al separar frontend y backend en dominios/puertos distintos: la petición "funciona" con `curl`, pero falla solo desde el navegador.

Cómo atienden peticiones por dentro (modelo de concurrencia)

- **Nginx**: modelo basado en eventos asíncronos; un número reducido de procesos atiende miles de conexiones a la vez sin bloquearse. Muy eficiente para servir estático y hacer de proxy.
- **Apache** (modo clásico): un proceso o hilo por conexión. Más sencillo de entender, algo más pesado con mucha concurrencia.
- **Node.js**: un único hilo con un *event loop*; funciona muy bien con operaciones de E/S (red, disco) que no bloquean ese hilo.
- **Gunicorn/Tomcat**: varios *workers* (procesos o hilos) en paralelo, cada uno atendiendo una petición a la vez.

¿Por qué importa el modelo de concurrencia?

- Determina cuántas peticiones simultáneas puede atender un servidor con los mismos recursos de CPU/RAM.
- Explica por qué Nginx se usa tan a menudo **delante** de servidores de aplicaciones: aguanta mucha más concurrencia sirviendo lo estático y repartiendo lo dinámico, mientras los workers del servidor de aplicaciones se dedican solo a ejecutar lógica.
- No hace falta memorizar los detalles internos: sí entender que "servidor web" y "servidor de aplicaciones" no solo se diferencian en qué sirven, sino en cómo están diseñados para servirlo.

Entonces... ¿se pueden usar los dos a la vez?

- Sí, es el patrón más habitual en producción: un **servidor web** (Nginx/Apache) por delante, que sirve directamente lo estático y **reenvía** (hace de proxy) las peticiones dinámicas a uno o varios **servidores de aplicaciones** por detrás.
- Esto se profundiza en la UD3 (proxy inverso) y en la UD4 (servidores de aplicaciones).

Esquema típico

```
navegador → Nginx (servidor web)
                ├── /css/*, /img/* → responde directamente (estático)
                └── /api/*         → reenvía a Node.js/Tomcat (servidor de aplicaciones)
                                       → ese código consulta la base de datos
                                       → genera la respuesta dinámica
```

Varios servidores de aplicaciones detrás: balanceo de carga

- Si un único servidor de aplicaciones no da abasto, se pueden levantar **varias instancias** (varios "backends") idénticas.
- El servidor web (o un balanceador dedicado) reparte las peticiones entre ellas, típicamente con una estrategia sencilla como **round robin** (una petición a cada backend por turnos).
- Existen otras estrategias de reparto y de despliegue gradual que solo citamos aquí, sin entrar en detalle (se amplían en UD3/UD5): **least connections**, **IP hash**, **weighted round robin**, **blue-green**, **canary**, **rolling**.
- Se profundiza en la UD3, pero conviene ya saber que "servidor de aplicaciones" no tiene por qué significar "una única instancia".

Escalado horizontal vs. vertical

- **Escalado vertical**: dar más recursos (CPU, RAM) a la misma máquina/instancia. Tiene un límite físico y, en algún momento, obliga a parar el servicio para ampliarlo.
- **Escalado horizontal**: añadir más instancias (más backends) en paralelo, como en la diapositiva anterior. Es el enfoque habitual en la nube: no tiene un límite tan rígido y encaja mejor con el balanceo de carga.
- Los contenedores (Docker) facilitan especialmente el escalado horizontal: levantar una instancia más es, en esencia, arrancar un contenedor más de la misma imagen.

De gestionar contenedores a mano a los orquestadores

- Arrancar, vigilar (health checks) y balancear manualmente docenas de contenedores no escala bien a mano.
- Los **orquestadores de contenedores** (el más conocido, **Kubernetes**) automatizan todo esto: deciden cuántas instancias levantar, las reinician si fallan, reparten el tráfico entre ellas y facilitan despliegues rolling/blue-green/canary de forma declarativa.
- No se usará en esta asignatura, pero es la evolución natural de "varios backends gestionados a mano" cuando eso se queda corto.

Health checks: ¿sigue vivo el backend?

- Antes de enviarle tráfico a un backend, el servidor web/balanceador puede comprobar periódicamente que sigue respondiendo correctamente (un **health check**: una petición de prueba a una ruta tipo `/health`).
- Si un backend deja de responder, se retira temporalmente de la rotación hasta que vuelva a estar sano, evitando que los usuarios reciban errores.

Por qué no servir todo directamente desde el servidor de aplicaciones

- Técnicamente se puede, pero los servidores web están mucho más optimizados para servir ficheros estáticos a gran velocidad y gestionar miles de conexiones simultáneas.
- Separar responsabilidades también facilita: cachear contenido estático, aplicar HTTPS en un único punto, y balancear carga entre varios servidores de aplicaciones.

Repaso — Sección 3, Pregunta 1

¿Cuál es la diferencia principal entre un servidor web y un servidor de aplicaciones?

A) No hay ninguna diferencia real, son sinónimos
B) El servidor web está optimizado para servir contenido (sobre todo estático) por HTTP; el servidor de aplicaciones ejecuta la lógica de negocio para generar contenido dinámico
C) El servidor de aplicaciones nunca puede usar HTTP
D) El servidor web solo funciona con bases de datos NoSQL

Repaso — Sección 3, Pregunta 1 (solución)

✅ **B)** El servidor web está optimizado para servir contenido (sobre todo estático) por HTTP; el servidor de aplicaciones ejecuta la lógica de negocio para generar contenido dinámico

Repaso — Sección 3, Pregunta 2

¿Para qué sirve un health check entre el servidor web/balanceador y los servidores de aplicaciones?

A) Para cifrar el tráfico entre ambos
B) Para comprobar periódicamente que un backend sigue respondiendo antes de enviarle tráfico de usuarios
C) Para generar el contenido dinámico directamente
D) Para resolver nombres de dominio

Repaso — Sección 3, Pregunta 2 (solución)

✅ **B)** Para comprobar periódicamente que un backend sigue respondiendo antes de enviarle tráfico de usuarios

Repaso — Sección 3, Pregunta 3

¿Por qué Nginx se usa tan a menudo delante de un servidor de aplicaciones como Tomcat o Node.js?

A) Porque Nginx no puede servir contenido estático
B) Porque su modelo de eventos asíncronos aguanta mucha concurrencia sirviendo estático y repartiendo lo dinámico, liberando al servidor de aplicaciones para centrarse en ejecutar lógica
C) Porque es obligatorio por ley usar ambos a la vez
D) Porque Tomcat no puede recibir peticiones HTTP directamente nunca

Repaso — Sección 3, Pregunta 3 (solución)

✅ **B)** Porque su modelo de eventos asíncronos aguanta mucha concurrencia sirviendo estático y repartiendo lo dinámico, liberando al servidor de aplicaciones para centrarse en ejecutar lógica

4. Qué significa "desplegar" una aplicación

Desplegar, en una frase

Desplegar es poner una aplicación a disposición de sus usuarios reales, ejecutándose en algún sitio accesible desde fuera de tu máquina de desarrollo.

Piezas que intervienen en un despliegue

- **Dominio**: el nombre (`miapp.com`) que los usuarios escriben.
- **DNS**: apunta ese dominio a la IP del servidor donde vive la app (visto en el bloque 2.1).
- **Hosting**: dónde se ejecuta físicamente la aplicación.
- **Certificado HTTPS**: para que la comunicación vaya cifrada (a fondo en la UD3).
- **Artefacto**: el "paquete" concreto y versionado de tu aplicación que realmente se despliega (siguiente apartado).

Tipos de hosting, de menos a más control

1. **Hosting compartido**: varias webs comparten un mismo servidor gestionado por el proveedor. Barato, poco control.
2. **VPS** (servidor privado virtual): una máquina virtual para ti solo, con control total del sistema operativo.
3. **Servidor dedicado**: una máquina física entera para ti.
4. **Cloud** (AWS, Azure, GCP…): recursos bajo demanda, escalables, facturados por uso.

Entornos de despliegue: no hay un único "producción"

- Es habitual tener varios **entornos** con el mismo propósito pero distinto nivel de exposición: **desarrollo** (tu máquina), **staging/pre-producción** (una réplica de producción para probar antes de publicar) y **producción** (el entorno real, con usuarios de verdad).
- Idea clave: el **mismo artefacto** versionado (la misma imagen Docker, el mismo `.jar`, el mismo `.tar.gz`) se **promociona** de un entorno a otro sin reconstruirlo, para garantizar que lo probado en staging es exactamente lo que llega a producción.
- Si cada entorno reconstruye su propio artefacto a partir del código, se pierde esa garantía: podrían colarse dependencias o versiones ligeramente distintas sin darse cuenta.

¿Qué es exactamente un "artefacto"?

- Un **artefacto** es el resultado empaquetado de construir tu aplicación: lo que de verdad se copia/instala/ejecuta en el servidor, no tu código fuente suelto.
- En lenguajes **compilados**, suele ser el binario o paquete compilado: un `.jar` / `.war` en Java, un `.exe`, un binario de Go...
- En lenguajes **interpretados** (que no se compilan a binario), el artefacto sigue existiendo: es un paquete con el código fuente ya listo para ejecutar junto con sus dependencias.

Artefactos en lenguajes que no se compilan: el caso de PHP

- PHP no se compila a un binario: el "artefacto" es, típicamente, un `.tar.gz` (o `.zip`) que contiene el código fuente de la aplicación **más** sus dependencias ya resueltas (la carpeta `vendor/` de Composer), listo para descomprimir en el servidor tal cual.
- La idea clave es la misma que en un lenguaje compilado: generar **una vez** un paquete inmutable, probarlo, y desplegar **exactamente ese mismo paquete**, en vez de "tirar de Git" directamente en producción o reinstalar dependencias en cada servidor (con el riesgo de que una reinstalación traiga una versión distinta sin querer).
- Otros ejemplos de artefactos de lenguajes interpretados: un paquete npm empaquetado (Node.js), una rueda `.whl` (Python), o directamente una **imagen Docker**, que empaqueta el artefacto junto con todo su entorno de ejecución.

Repositorios de artefactos

- Igual que **Docker Hub** almacena y distribuye imágenes (UD1), existen repositorios equivalentes para artefactos de cualquier tipo: **JFrog Artifactory**, **Sonatype Nexus**, o los registries de paquetes de cada lenguaje (npm, PyPI...).
- Permiten subir una vez el artefacto generado (con su número de versión) y descargar **siempre la misma versión exacta** desde cualquier servidor, en vez de reconstruirlo cada vez.
- También sirven de "fuente de verdad": si hay un incidente, puedes comprobar exactamente qué artefacto (qué versión) está desplegado y volver a descargar esa misma versión para reproducir el problema.

La importancia de versionar bien los artefactos

- Cada build relevante debería generar un artefacto con un **número de versión** propio, siguiendo un esquema claro como **SemVer** (`MAJOR.MINOR.PATCH`, p. ej. `2.4.1`): cambios incompatibles incrementan `MAJOR`, nuevas funcionalidades compatibles incrementan `MINOR`, correcciones incrementan `PATCH`.
- Ya vimos en la UD1 el problema de usar el tag `latest` en Docker: aquí es exactamente la misma idea. Desplegar siempre "la última" versión sin fijarla hace que no sea reproducible saber qué hay exactamente en producción.
- Versionar explícitamente permite: saber qué hay desplegado en cada entorno, repetir exactamente el mismo despliegue en otra máquina, y hacer **rollback** (volver atrás) a una versión anterior conocida si algo falla.

Configuración y secretos: lo único que debería cambiar entre entornos

- Si el artefacto es el mismo en todos los entornos (diapositiva anterior sobre entornos), ¿cómo sabe la aplicación a qué base de datos conectarse en cada uno, o qué credenciales usar?
- Mediante **configuración externa al artefacto**: variables de entorno, ficheros de configuración montados aparte, o un gestor de secretos dedicado (Vault, AWS Secrets Manager...).
- Regla importante: las contraseñas, tokens o claves de API **nunca** deberían quedar hardcodeadas dentro del código ni "horneadas" dentro de la imagen/artefacto — así el mismo artefacto sirve para todos los entornos, cambiando solo su configuración externa.

Despliegue con contenedores

- Ya conoces **Docker**: empaquetar la aplicación en una imagen y ejecutarla como contenedor es, hoy en día, una de las formas más habituales de desplegar, y la imagen Docker es, en sí misma, un tipo de artefacto versionado (por su tag).
- Ventaja clave que ya vimos en la UD1: el mismo contenedor que probaste en tu portátil es, en esencia, el que se ejecuta en el servidor de producción.
- Esto se automatiza más adelante con CI/CD (UD5): cada cambio de código puede acabar generando y desplegando un nuevo artefacto (imagen) versionado automáticamente.

Un flujo de despliegue simple (sin entrar en CI/CD todavía)

```
1. Tienes el código (y su Dockerfile) en tu repositorio Git
2. Construyes el artefacto: docker build, etiquetado con una versión (p. ej. miapp:1.4.0)
3. Subes el artefacto a un repositorio (registry de Docker, o Artifactory/Nexus si no fuera Docker)
4. En el servidor: docker pull miapp:1.4.0 + docker run (o docker compose up)
5. El servidor web (Nginx) por delante enruta el tráfico hacia el contenedor
```

Repaso — Sección 4, Pregunta 1

¿Qué es, en el contexto del despliegue, un "artefacto"?

A) Únicamente un binario compilado, nunca puede ser otra cosa
B) El resultado empaquetado y versionado de construir la aplicación, listo para desplegar
C) Un sinónimo exacto de "dominio"
D) El nombre que recibe siempre el servidor de producción

Repaso — Sección 4, Pregunta 1 (solución)

✅ **B)** El resultado empaquetado y versionado de construir la aplicación, listo para desplegar

Repaso — Sección 4, Pregunta 2

PHP no se compila a un binario. ¿Cómo se genera típicamente su artefacto de despliegue?

A) Es imposible generar un artefacto de una aplicación PHP
B) Un `.tar.gz` / `.zip` con el código fuente y sus dependencias (p. ej. `vendor/` de Composer) ya resueltas
C) Siempre como una imagen de máquina virtual completa
D) PHP solo puede desplegarse copiando ficheros manualmente uno a uno sin empaquetar nada

Repaso — Sección 4, Pregunta 2 (solución)

✅ **B)** Un `.tar.gz` / `.zip` con el código fuente y sus dependencias (p. ej. `vendor/` de Composer) ya resueltas

Repaso — Sección 4, Pregunta 3

¿Por qué es importante versionar explícitamente los artefactos en vez de desplegar siempre "la última" versión disponible?

A) No es importante, da exactamente igual qué versión se despliega
B) Porque permite saber con certeza qué hay desplegado, repetir el despliegue de forma reproducible y poder hacer rollback si algo falla
C) Porque las versiones antiguas ocupan menos espacio en disco
D) Porque solo las versiones con nombre numérico pueden ejecutarse

Repaso — Sección 4, Pregunta 3 (solución)

✅ **B)** Porque permite saber con certeza qué hay desplegado, repetir el despliegue de forma reproducible y poder hacer rollback si algo falla

5. Todo junto: de la petición al despliegue

El panorama completo del bloque 2.2

```
usuario escribe miapp.com
   → DNS resuelve la IP del servidor
   → llega al servidor web (Nginx), que escucha en el puerto 443 (HTTPS)
       ├── pide /css/estilo.css → Nginx lo sirve directo, o lo sirve una CDN por delante (estático)
       └── pide /api/productos  → Nginx reenvía a uno de los backends sanos
                                    con el servidor de aplicaciones (dinámico)
                                    → consulta la base de datos
                                    → devuelve JSON
   → Nginx devuelve la respuesta final al navegador
```

¿Qué artefacto concreto hay corriendo ahí detrás?

- Cada backend de ese esquema está ejecutando **una versión concreta** de un artefacto (p. ej. la imagen Docker `miapp:1.4.0`), descargada de un repositorio de artefactos.
- Saber exactamente qué versión está desplegada en cada entorno (desarrollo, pruebas, producción) es lo que permite diagnosticar incidencias y, si hace falta, volver atrás con confianza.

Desplegar una nueva versión sin cortar el servicio

Si ya tienes usuarios reales, no puedes simplemente "apagar y encender" para actualizar. Dos ideas habituales (se profundizan en UD5):

- **Despliegue continuo/rolling**: se van sustituyendo los backends antiguos por los nuevos de uno en uno, mientras los demás siguen atendiendo tráfico.
- **Blue-green**: se levanta en paralelo un entorno completo con la nueva versión ("green"); cuando está verificado, el servidor web pasa a enrutar todo el tráfico hacia él, dejando el antiguo ("blue") como respaldo inmediato para un rollback rápido.

Feature flags: separar "desplegar" de "lanzar"

- Una **feature flag** (interruptor de funcionalidad) es una condición en el código que activa o desactiva una funcionalidad sin necesidad de desplegar de nuevo.
- Permite **desplegar** código nuevo en producción ya apagado, y **lanzarlo** después (activando el flag) cuando se decida, incluso solo para un grupo reducido de usuarios — una forma ligera de conseguir algo parecido a un canary sin depender de la infraestructura de balanceo.
- Si algo falla tras activar la funcionalidad, basta con apagar el flag: es un rollback casi instantáneo, sin tocar el despliegue en sí.

El rollback no siempre es gratis: migraciones de base de datos

- Si la nueva versión del artefacto cambia el **esquema** de la base de datos (añade una columna, cambia un tipo…), no basta con volver a desplegar el artefacto anterior para hacer rollback: la base de datos ya ha cambiado.
- Por eso las migraciones se diseñan, siempre que se pueda, para ser **compatibles hacia atrás** durante un tiempo (p. ej. añadir una columna opcional en vez de renombrar una existente), de forma que la versión antigua y la nueva del código puedan convivir con el mismo esquema mientras dura el despliegue.
- Se trabajará con más detalle más adelante; hoy basta con saber que "desplegar código" y "migrar datos" son cosas relacionadas pero distintas, y que un rollback de código no deshace automáticamente un cambio de esquema ya aplicado.

Monitorización y rollback

- Tras desplegar, conviene vigilar métricas básicas (errores 5xx, tiempo de respuesta) para detectar pronto si algo ha ido mal.
- Si el artefacto nuevo falla, tener la versión anterior bien identificada en el repositorio de artefactos permite un **rollback** rápido: simplemente volver a desplegar la versión anterior conocida, en vez de intentar arreglar en caliente.

Por qué esto no es "una caja negra"

- Cada pieza de este esquema es algo que ya conoces o vas a tocar pronto: DNS y HTTP (bloque 2.1), Docker (UD1), Nginx/Apache en detalle (UD3), servidores de aplicaciones en detalle (UD4), automatización del despliegue (UD5).
- Entender el esquema completo ahora te va a servir de mapa para todo lo que viene después.

Qué viene después

- **UD3**: configurar Apache/Nginx de verdad (hosts virtuales, HTTPS, proxy inverso, rendimiento).
- **UD4**: servidores de aplicaciones en profundidad (Tomcat y similares) y transferencia de ficheros.
- **UD5**: automatizar todo este flujo con CI/CD, incluyendo la generación y versionado automático de artefactos, y estrategias de despliegue como rolling o blue-green.

Repaso — Sección 5, Pregunta 1

En el esquema completo visto hoy, ¿qué hace Nginx con una petición a `/css/estilo.css`?

A) La reenvía siempre al servidor de aplicaciones

B) La sirve directamente (o la sirve una CDN por delante), al ser contenido estático

C) La rechaza con un error 403

D) Consulta la base de datos antes de responder

Repaso — Sección 5, Pregunta 1 (solución)

✅ **B)** La sirve directamente (o la sirve una CDN por delante), al ser contenido estático

Repaso — Sección 5, Pregunta 2

¿Qué diferencia principal hay entre un despliegue "rolling" y uno "blue-green"?

A) No hay ninguna diferencia real

B) En rolling se sustituyen los backends de uno en uno; en blue-green se levanta un entorno nuevo completo en paralelo y se cambia el tráfico de golpe cuando está listo

C) Blue-green solo puede usarse con hosting compartido

D) Rolling nunca permite hacer rollback

Repaso — Sección 5, Pregunta 2 (solución)

✅ **B)** En rolling se sustituyen los backends de uno en uno; en blue-green se levanta un entorno nuevo completo en paralelo y se cambia el tráfico de golpe cuando está listo

Repaso — Sección 5, Pregunta 3

¿Por qué tener los artefactos bien versionados facilita hacer un rollback tras un despliegue fallido?

A) Porque permite volver a desplegar, con confianza, la versión anterior conocida que funcionaba bien
B) Porque los artefactos versionados ocupan menos espacio automáticamente
C) Porque el versionado elimina la necesidad de monitorización
D) Porque un rollback solo es posible sin versionado

Repaso — Sección 5, Pregunta 3 (solución)

✅ **A)** Porque permite volver a desplegar, con confianza, la versión anterior conocida que funcionaba bien

A practicar

Servidores web, de aplicaciones y despliegue